

iOS 发布步骤 (Archive / IPA / Upload)

本文档记录当前仓库可复现的 iOS 发布流程，覆盖：

- 归档 `xcarchive`
- 导出或手工封装 `ipa`
- 上传到 App Store Connect
- 查询上传后的处理状态

0. 一键脚本

优先使用仓库脚本：

```
scripts/release_ios.sh
```

常用变体：

```
scripts/release_ios.sh --skip-upload  
scripts/release_ios.sh --wait  
scripts/release_ios.sh --build-number 202603101745
```

说明：

- 默认串行执行 `archive -> export/兜底封包 -> upload`
- `--skip-upload` 只生成 `ipa`，便于先本地检查
- `--wait` 会在上传后继续等待 App Store Connect 处理完成
- 如果 `pubspec.yaml` 没有显式 `+build-number`，脚本会自动生成时间戳 `build number`，避免重复上传被拒
- 可通过环境变量覆盖 `APP_STORE_API_KEY_PATH`、`APP_STORE_API_KEY_ID`、`APP_STORE_API_ISSUER_ID`、`IOS_TEAM_ID`、`IOS_BUILD_NAME`、`IOS_BUILD_NUMBER`

当前文档基于以下已确认信息：

- Bundle ID：`top.echo-loop`
- Team ID：`S8S968QAV3`
- App Store Connect API Key：
- Issuer ID：`3ec439fe-b66c-4034-b8c2-16e133fc4d6b`
- Key ID：`5GB5KL75VZ`
- 私钥文件：`ios/AuthKey_5GB5KL75VZ.p8`

1. 前置条件

发布前先确认：

- 已在 App Store Connect 创建 App，Bundle ID 选择 `top.echo-loop`
- 本机可正常使用 Xcode、CocoaPods、Flutter
- `ios/AuthKey_5GB5KL75VZ.p8` 已放到仓库 `ios/` 目录
- 当前代码已经是准备发布的版本

建议先执行仓库预检：

```
scripts/preflight.sh
```

作用：

- 确认当前分支、工作区改动、关键文件是否存在

2. 检查版本号

iOS 的版本号来自 Flutter：

- `CFBundleShortVersionString` ← `pubspec.yaml` 的 `version` 主版本号
- `CFBundleVersion` ← Flutter build number

先看当前版本：

```
rg -n "^version:" pubspec.yaml
```

作用：

- 确认本次准备上传的版本号

如果要显式指定版本号和构建号，也可以在构建时传：

```
flutter build ios --release --build-name 1.0.1 --build-number 1.0.1
```

作用：

- 覆盖默认版本号，避免重复上传相同 build number

3. 检查本机签名身份

```
security find-identity -v -p codesigning
```

作用：

- 查看本机钥匙串里当前可用的签名证书

说明：

- 归档阶段即使只有 `Apple Development`，也可能先成功

- 导出 App Store 包时仍然需要 Apple Distribution
- 如果本机没有 Apple Distribution 私钥，Xcode 可能会尝试通过 API Key 在苹果后台创建证书和描述文件

4. 生成 **ExportOptions.plist**

先准备一个导出配置文件：

```
cat > /tmp/fluency_export_options.plist <<'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>destination</key>
  <string>export</string>
  <key>manageAppVersionAndBuildNumber</key>
  <false/>
  <key>method</key>
  <string>app-store</string>
  <key>signingStyle</key>
  <string>automatic</string>
  <key>stripSwiftSymbols</key>
  <true/>
  <key>teamID</key>
  <string>S8S968QAV3</string>
  <key>uploadSymbols</key>
  <true/>
</dict>
</plist>
EOF
```

作用：

- 告诉 Xcode 用自动签名方式导出 App Store 包

说明：

- 当前 Xcode 会提示 app-store 已废弃，推荐改为 app-store-connect
- 但本仓库这次实际验证时，app-store 仍可继续执行导出流程

5. 生成 **xcarchive**

```
xcodebuild \
-workspace ios/Runner.xcworkspace \
-scheme Runner \
-configuration Release \
-destination generic/platform=iOS \
-archivePath /tmp/fluency-release.xcarchive \
-allowProvisioningUpdates \
```

```
-authenticationKeyPath "$PWD/ios/AuthKey_5GB5KL75VZ.p8" \  
-authenticationKeyID 5GB5KL75VZ \  
-authenticationKeyIssuerID 3ec439fe-b66c-4034-b8c2-16e133fc4d6b \  
archive
```

作用：

- 用 Xcode 正式归档 iOS App
- 允许 Xcode 自动拉取或修复描述文件
- 用 App Store Connect API Key 访问苹果签名服务

成功标志：

- 输出末尾出现 `** ARCHIVE SUCCEEDED **`

6. 常规导出 IPA

```
xcodebuild \  
-exportArchive \  
-archivePath /tmp/fluency-release.xcarchive \  
-exportPath /tmp/fluency-export \  
-exportOptionsPlist /tmp/fluency_export_options.plist \  
-allowProvisioningUpdates \  
-authenticationKeyPath "$PWD/ios/AuthKey_5GB5KL75VZ.p8" \  
-authenticationKeyID 5GB5KL75VZ \  
-authenticationKeyIssuerID 3ec439fe-b66c-4034-b8c2-16e133fc4d6b
```

作用：

- 将 `xcarchive` 导出为可上传的 `ipa`

正常情况下导出结果：

- `ipa` 在 `/tmp/fluency-export/`

7. 当前机器的兜底方案：手工封装 IPA

这次实测中，`xcodebuild -exportArchive` 在最后一步失败：

- 错误：`Copy failed`
- 根因：Xcode 26 导出阶段调用 `/usr/bin/rsync` 时触发 `--extended-attributes` 兼容性问题

但在失败前，Xcode 已经完成了正式分发签名，因此可以直接回收临时目录里的 `Payload` 手工打包。

先确认临时导出的 `Runner.app` 是否是正式分发签名：

```
codesign -dv --verbose=4 \  
/var/folders/8f/vmldyl4s2kb9nv6q34gn37rm0000gn/T/XcodeDistPipeline.~~~z3J41b/Root/Payload/Runner.app \  
2>&1 | sed -n '1,120p'
```

作用：

- 确认签名主体是否为 Apple Distribution: ...

如果确认已经是 Distribution 签名，再手工封装：

```
mkdir -p /tmp/fluency-export
cd /var/folders/8f/vmldyl4s2kb9nv6q34gn37rm0000gn/T/XcodeDistPipeline.~~~z3J41b/Root
/usr/bin/ditto -c -k --sequesterRsrc --keepParent Payload /tmp/fluency-export/Runner.ipa
ls -lh /tmp/fluency-export/Runner.ipa
```

作用：

- 直接把已经签名完成的 Payload/Runner.app 打成 ipa

说明：

- XcodeDistPipeline.~~~xxxxxx 的目录名每次都不同，实际执行时先用 `find /var/folders/.../T -maxdepth 1 -type d -name 'XcodeDistPipeline.*'` 找最新目录

8. 上传到 App Store Connect

推荐显式传入 .p8 路径，避免 altool 在默认目录找不到私钥：

```
xcrun altool \
  --upload-app \
  --type ios \
  --file /tmp/fluency-export/Runner.ipa \
  --apiKey 5GB5KL75VZ \
  --apiIssuer 3ec439fe-b66c-4034-b8c2-16e133fc4d6b \
  --p8-file-path "$PWD/ios/AuthKey_5GB5KL75VZ.p8" \
  --output-format json \
  --show-progress
```

作用：

- 将 Runner.ipa 上传到 App Store Connect

成功标志：

- 输出 Upload succeeded.
- 返回 Delivery UUID

本次实测上传成功返回：

- Delivery UUID: 0bf3b055-cde0-4e33-8b65-ccc5b92fbcec

9. 查询上传后的处理状态

拿到 Delivery UUID 后，可以继续查处理状态：

```
xcrun altool \
  --build-status \
```

```
--delivery-id 0bf3b055-cde0-4e33-8b65-ccc5b92fbcec \  
--apiKey 5GB5KL75VZ \  
--apiIssuer 3ec439fe-b66c-4034-b8c2-16e133fc4d6b \  
--p8-file-path "$PWD/ios/AuthKey_5GB5KL75VZ.p8" \  
--output-format json
```

作用：

- 查询这次上传是否还在 `processing`
- 确认是否已经进入 App Store Connect 构建列表

如果希望命令阻塞等待处理完成，可以追加：

```
--wait
```

10. 这次上传出现的 **warning**

本次上传成功，但 Apple 返回了 2 个 warning：

- WebVTT Subtitle 缺少 `LSHandlerRank`
- SubRip Subtitle 缺少 `LSHandlerRank`

对应文件：

- `ios/Runner/Info.plist`

对应位置：

- `CFBundleDocumentTypes`

这两个 warning 不阻塞上传。

说明：

- 当前仓库已经补上两个文档类型的 `LSHandlerRank = Alternate`
- 下次重新上传时，这两条 warning 预期应当消失

11. 常用排查命令

查看 Xcode Release 配置：

```
xcodebuild -showBuildSettings -project ios/Runner.xcodeproj -scheme Runner -configuration Release
```

作用：

- 确认 Team ID、Bundle ID、签名方式、版本号

查看 App 关键 plist 配置：

```
/usr/libexec/PlistBuddy -c 'Print :CFBundleIdentifier' ios/Runner/Info.plist  
/usr/libexec/PlistBuddy -c 'Print :CFBundleDisplayName' ios/Runner/Info.plist
```

```
/usr/libexec/PlistBuddy -c 'Print :CFBundleShortVersionString' ios/Runner/Info.plist
/usr/libexec/PlistBuddy -c 'Print :CFBundleVersion' ios/Runner/Info.plist
```

作用：

- 快速确认包名、显示名、版本配置入口

12. 建议的完整顺序

推荐按这个顺序执行：

1. `scripts/preflight.sh`
2. 检查 `pubspec.yaml` 版本号
3. `security find-identity -v -p codesigning`
4. 生成 `/tmp/fluency_export_options.plist`
5. `xcodebuild ... archive`
6. `xcodebuild -exportArchive ...`
7. 如果导出失败但签名已完成，手工 `ditto` 打包 `ipa`
8. `xcrun altool --upload-app ...`
9. `xcrun altool --build-status ...`

13. 相关文档

- [docs/ios-app-store-checklist.md](#)
- [docs/ios-universal-links.md](#)